

PaLM: Scaling Language Modeling with Pathways

Chowdhery, Aakanksha, et al. *arXiv preprint arXiv:2204.02311*

2023. 02. 19

허정원, 조해창, 박산희



Contents

- Introduction
- Model Architecture
- Training
- Evaluation
- Discussions



1. Introduction

Over the last few years, very large neural networks trained for language understanding and generation have achieved astonishing results across a wide array of tasks.



175B



GaLM

1.2T



LaMDA

137B



Gopher

280B



MT NLG

530B



1. Introduction

The improvements have primarily come from one or more of the following approaches:

- (1) Scaling the size of the models in both depth and width
- (2) Increasing the number of tokens that the model was trained on
- (3) Training on cleaner datasets from more diverse sources
- (4) Increasing model capacity without increasing the computational cost through sparsely activated modules.



PaLM

540B
780B Tokens

Achieved through the use of Pathways



The key takeaways

- Efficient scaling Section 4
- Continued improvements from scaling Section 6
- Breakthrough capabilities Section 6.2 6.3 9
- Discontinuous improvements Section 6.2
- Multilingual understanding Section 6.5 6.6 6.7
- Bias and toxicity Section 10

Model Architecture



2. Model Architecture

Standard Transformer model architecture in a decoder-only setup

Attention is all you need

- SwiGLU Activation
- Parallel Layers
- Multi-Query Attention
- RoPE Embeddings
- Shared Input-Output Embeddings
- No Biases
- Vocabulary

GLU variants improve transformer

A 6 Billion Parameter Autoregressive Language Model

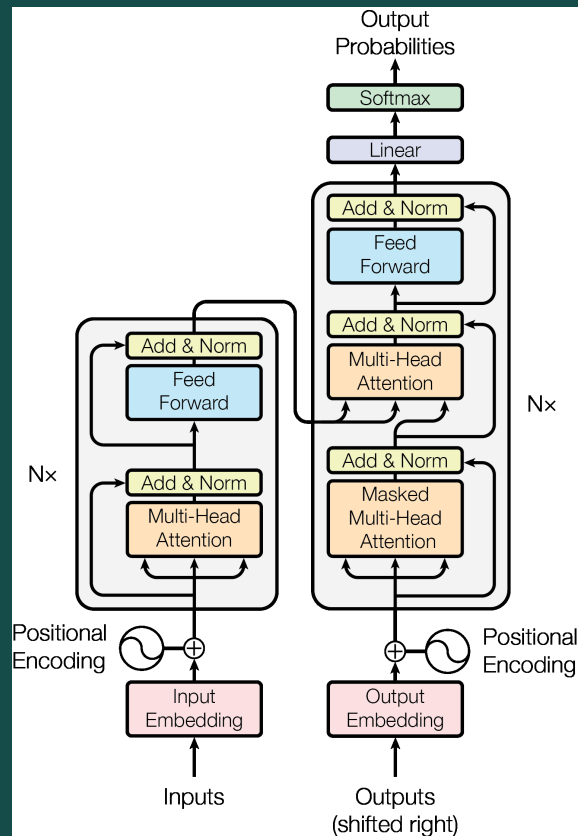
One write-head is all you need

Roformer: Enhanced transformer with rotary position embedding

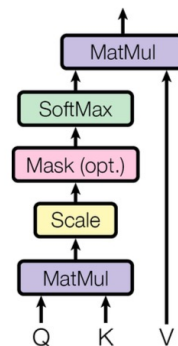
SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing

2. Model Architecture

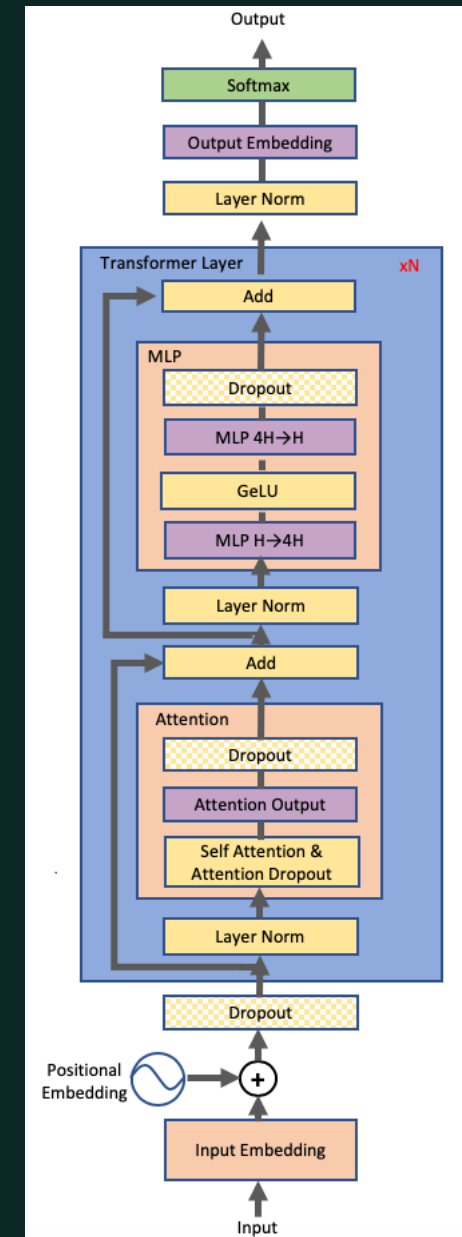
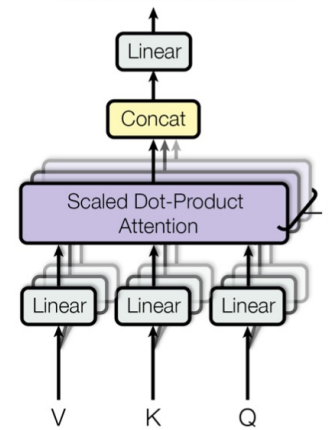
A decoder-only setup



Scaled Dot-Product Attention

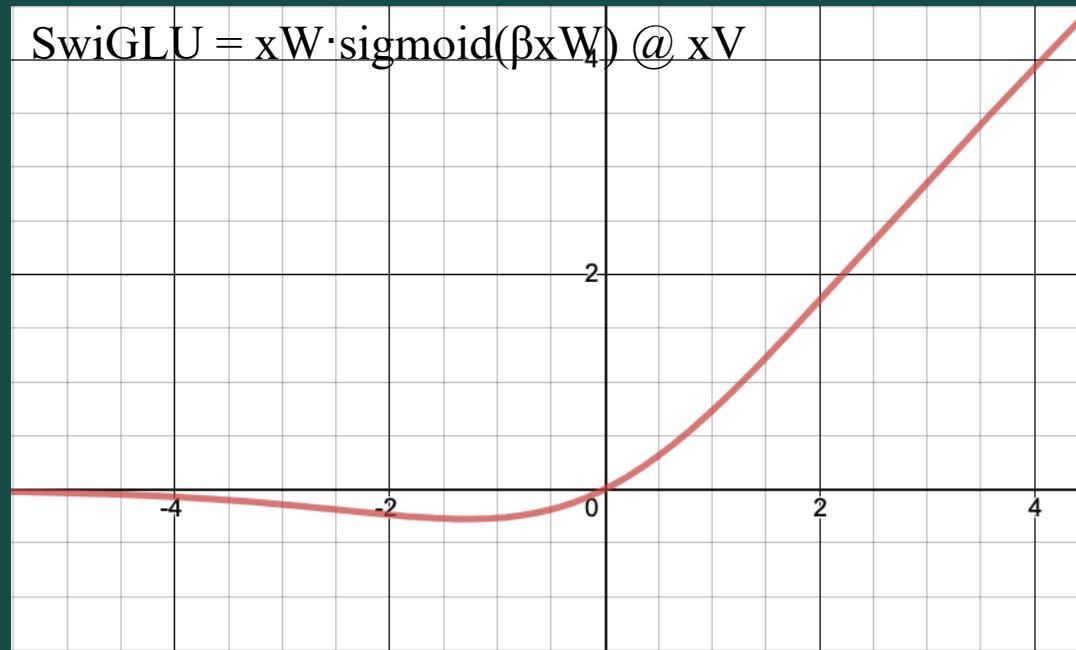


Multi-Head Attention

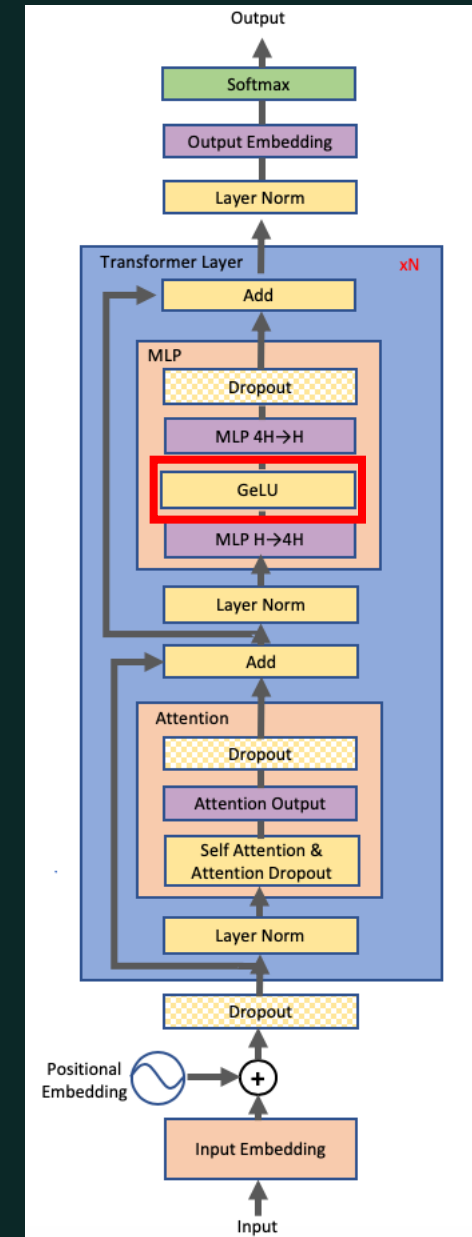


2. Model Architecture

- **SwiGLU Activation**



An improvement in quality in compute- equivalent experiments

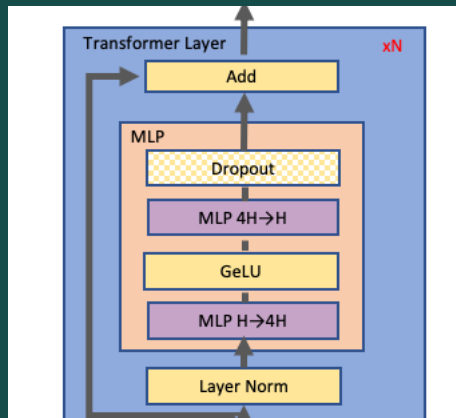
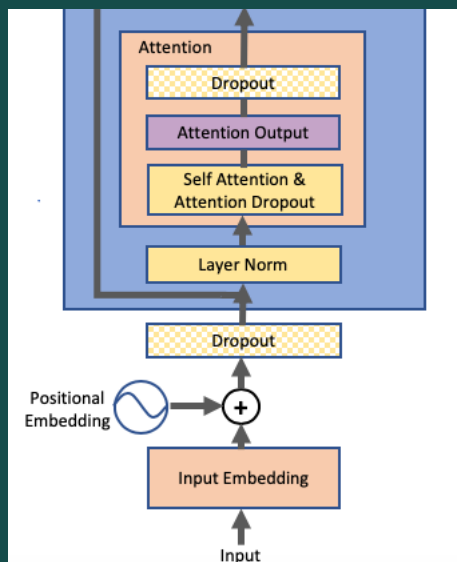


2. Model Architecture

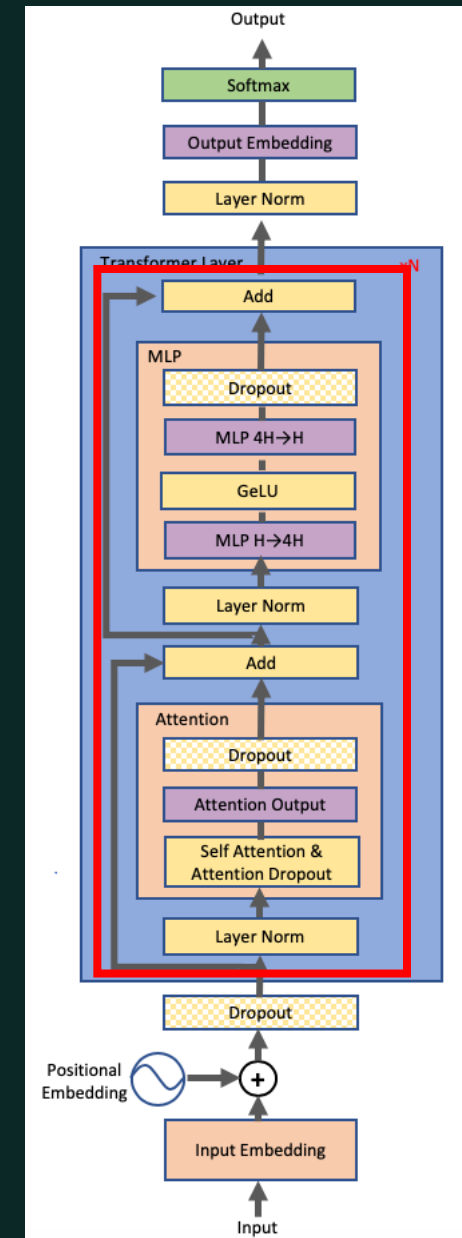
- **Parallel Layers**

$$y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x))))$$

$$y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$$



The parallel formulation results in roughly 15% faster training speed at large scales, since the MLP and Attention input matrix multiplications can be fused.



2. Model Architecture

- **Multi-Query Attention**

k: number of attention heads

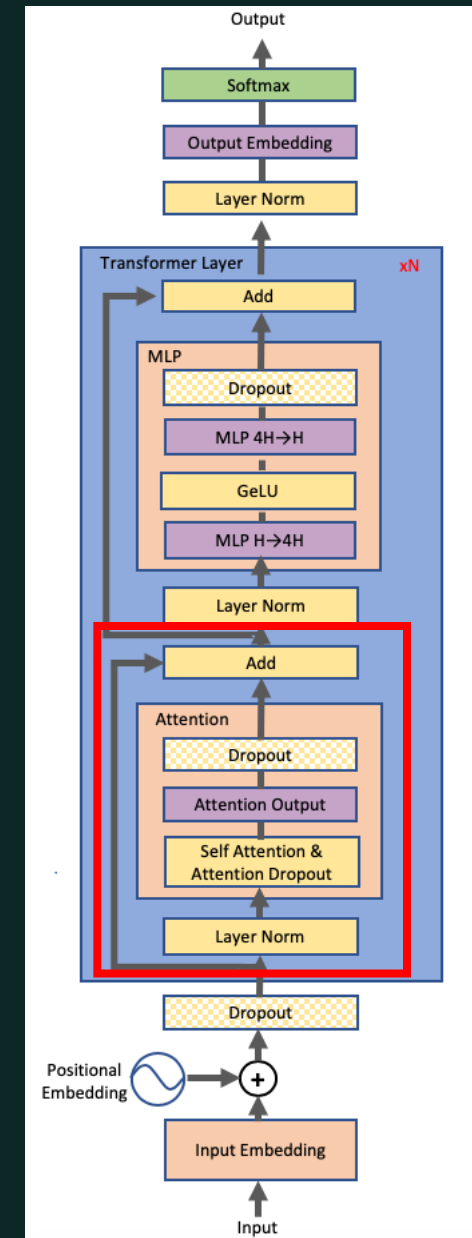
h: attention head size

$[k, h]$

Key/Value projections are shared for each head $\rightarrow [1, h]$

Query is still projected to shape $\rightarrow [k, h]$

This has a neutral effect on model quality and training speed, but results in a significant cost savings at autoregressive decoding time.



2. Model Architecture

- **RoPE Embeddings**

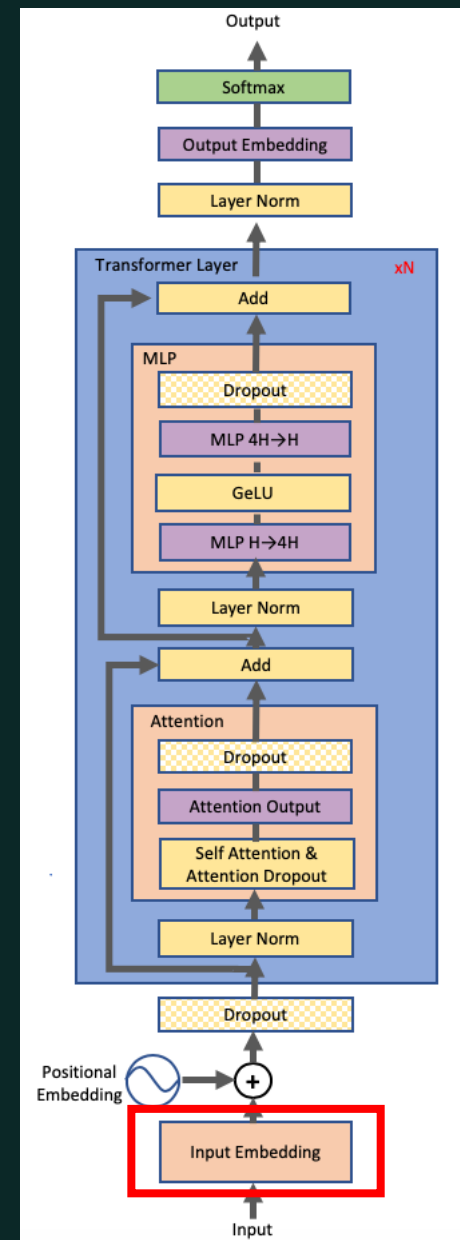
$$\text{PE}_{(\text{pos}, 2i)} = \sin(\text{pos}/10000^{2i/d_{\text{model}}})$$

$$\text{PE}_{(\text{pos}, 2i+1)} = \cos(\text{pos}/10000^{2i/d_{\text{model}}})$$

Incorporates explicit relative position dependency in self-attention to enhance the performance of transformer architectures

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$

$$\begin{aligned} f_q(x_m) &:= W_q x_m \\ f_k(x_n + n) &:= W_k(x_n + \tilde{p}_r^k) \\ f_v(x_n + n) &:= W_v(x_n + \tilde{p}_r^v) \end{aligned}$$

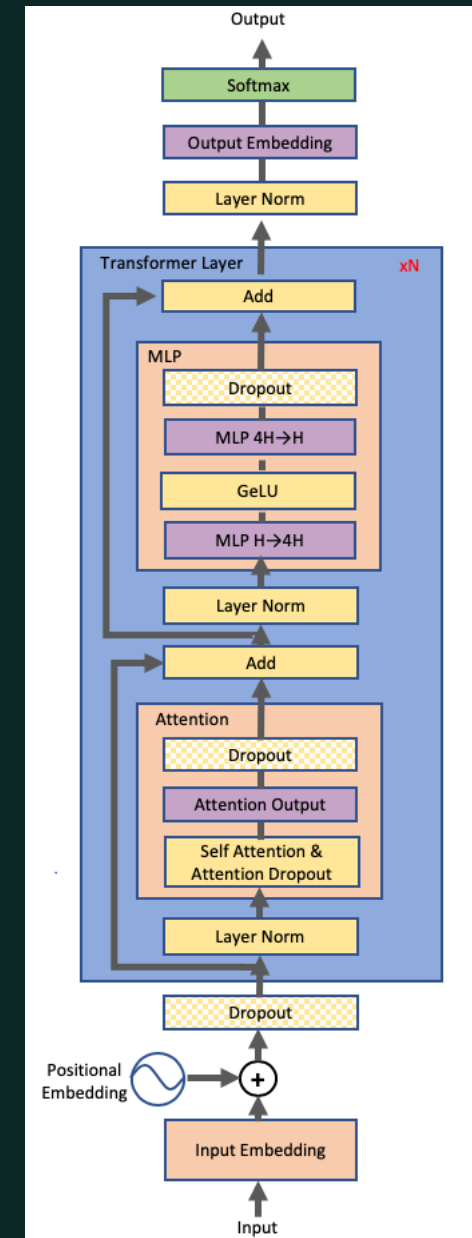


2. Model Architecture

- **Vocabulary**

A SentencePiece vocabulary with 256k tokens, which was chosen to support the large number of languages in the training corpus without excess tokenization.

The vocabulary is completely lossless and reversible.





2. Model Architecture

Standard Transformer model architecture in a decoder-only setup

- SwiGLU Activation → significantly increase **quality**
- Parallel Layers → results in roughly 15% **faster**
- Multi-Query Attention → significant **cost savings** at decoding time
- RoPE Embeddings → better **performance** on long sequence
- Shared Input-Output Embeddings
- No Biases → increased training **stability**
- Vocabulary → improves training **efficiency**



2.1 Model Scale Hyperparameters

540B parameters, 62B parameters, and 8B parameters.

Model	Layers	# of Heads	d_{model}	# of Parameters (in billions)	Batch Size
PaLM 8B	32	16	4096	8.63	256 → 512
PaLM 62B	64	32	8192	62.50	512 → 1024
PaLM 540B	118	48	18432	540.35	512 → 1024 → 2048

The feed-forward size d_{ff} is always $4 \times d_{\text{model}}$ and attention head size is always 256.

Model Architecture

Q & A

Training



3 Training Dataset

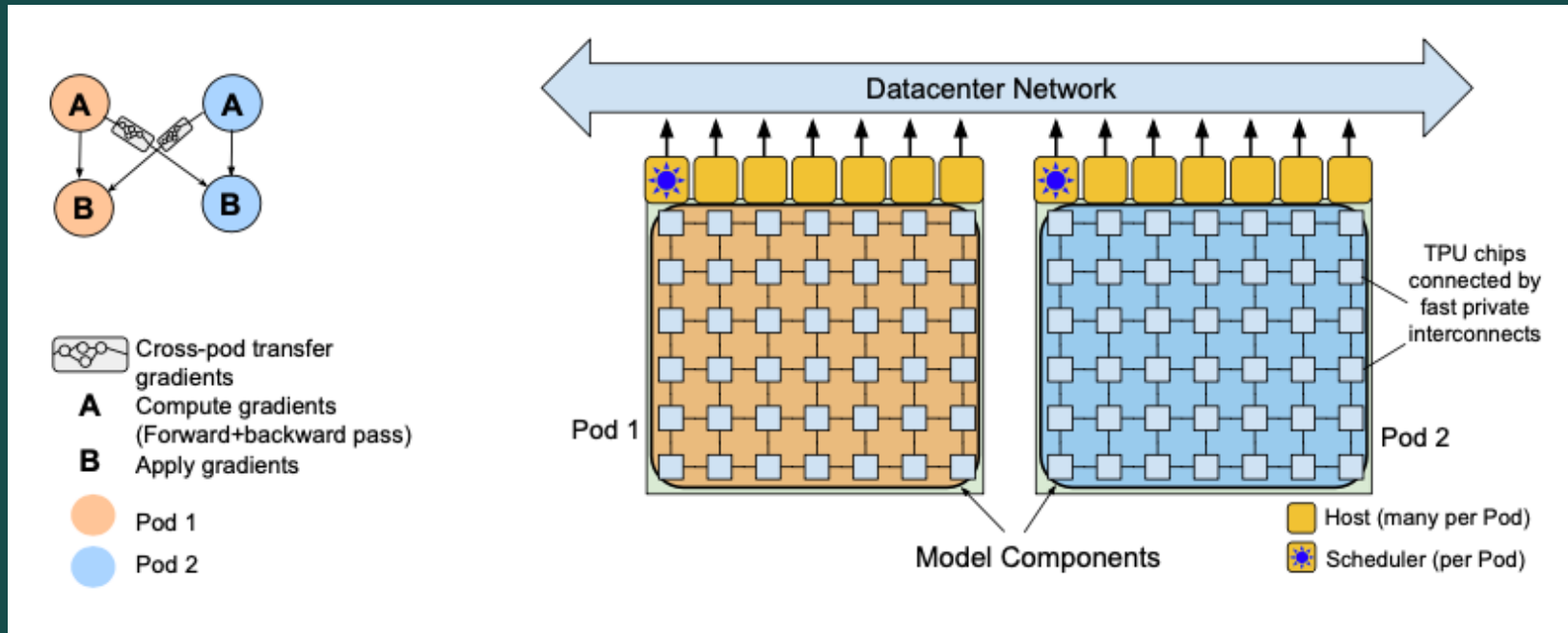
A corpus of 780 billion tokens that represent a wide range of natural language use cases.

The dataset is a mixture of filtered webpages, books, Wikipedia, news articles, source code, and social media conversations.

Total dataset size = 780 billion tokens	
Data source	Proportion of data
Social media conversations (multilingual)	50%
Filtered webpages (multilingual)	27%
Books (English)	13%
GitHub (code)	5%
Wikipedia (multilingual)	4%
News (English)	1%



4 Training Infrastructure



The client-server arch of Pathways to achieve two-way data parallelism at the pod level. Pipelining is typically used with data center network (DCN).



4.1 Training Efficiency

Model	# of Parameters (in billions)	Accelerator chips	Model FLOPS utilization
GPT-3	175B	V100	21.3%
Gopher	280B	4096 TPU v3	32.5%
Megatron-Turing NLG	530B	2240 A100	30.2%
PaLM	540B	6144 TPU v4	46.2%

The number of hardware FLOPs executed is system-dependent and implementation-dependent.

FLOPs is dependent on methodology used to count or track them.

Model FLOPs Utilization (MFU)



5 Training Setup

- Weight initialization
- Optimizer Adafactor: Adaptive learning rates with sublinear memory cost.
- Optimization hyperparameters
- Loss function
- Sequence length
- Batch size Don't decay the learning rate, increase the batch size.
- Bitwise determinism Scaling up models and data with t5x and seqio.
- Dropout



5 Training Setup

- **Weight initialization**

The Kernel weights are initialized with “fan-in variance scaling”

$$W \sim N(0, 1 / \sqrt{n_{in}})$$

The input embeddings are initialized to

$$E \sim N(0, 1)$$



5 Training Setup

- **Optimizer**

Adafactor optimizer without factorization

Effectively equivalent to Adam with “parameter scaling”

Parameter matrices which operate at different scales do not have their learning rate scaled down at the same rate.



5 Training Setup

- **Optimization hyperparameters**

An adafactor learning rate of 10^{-2} for the first 10,000 steps, which is then decayed at a rate of $1/\sqrt{k}$, where k is step number.

Momentum of $\beta_1 = 0.9$,

The second-order moment interpolation value is computed as $\beta_2 = 1.0 - k^{-0.8}$

More stable than the standard $\beta_2 = 0.99$, because rare embedding tokens can have poorly estimated second moments over shorter windows.

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \end{aligned}$$



5 Training Setup

- **Loss function**

Average log probability of all tokens without label smoothing

$$Loss = -1/n \cdot \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

An auxiliary loss of $z \text{ loss} = 10^{-4} \cdot \log Z^2$ to encourage the softmax normalizer $\log(Z)$ to be close to 0, which they found increases the stability of training.



5 Training Setup

- **Batch size**

512(~50k) → 1024(~115k) → 2048

Smaller batch sizes are more sample efficient earlier in training.

Larger batch sizes are beneficial later in training due to better gradient estimates.

Larger batch sizes increases TPU efficiency.



5 Training Setup

- **Sequence length**

Input examples are concatenated together and then split into sequences of exactly 2048 tokens.

- **Bitwise determinism**

JAX+XLA+T5X

The shuffled data is written out in a random-access format.

- **Dropout**

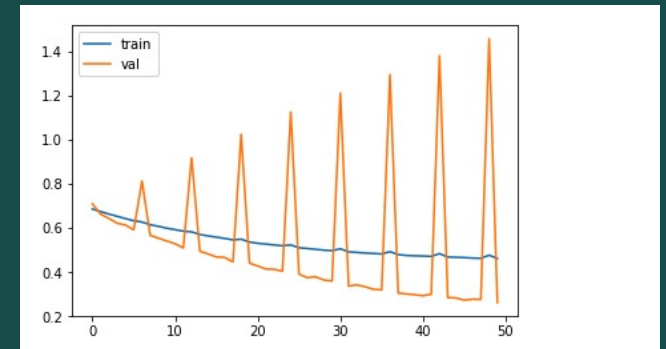
The model was trained without dropout.



5.1 Training Instability

They observed spikes in the loss roughly 20 times during training.

They re-started training and skipped roughly 200–500 data batches. With this mitigation, the loss did not spike again at the same point.



The spikes were “**not**” caused by “bad data” per se, because they ran several ablation experiments and they did not see a spike.

This implies that spikes only occur due to the combination of specific data batches with a particular model parameter state.

Training

Q & A

Evaluation

6.1 English NLP tasks

Evaluate the PaLM model on the same set of 29 English benchmarks as **GLaM**, **GPT-3**

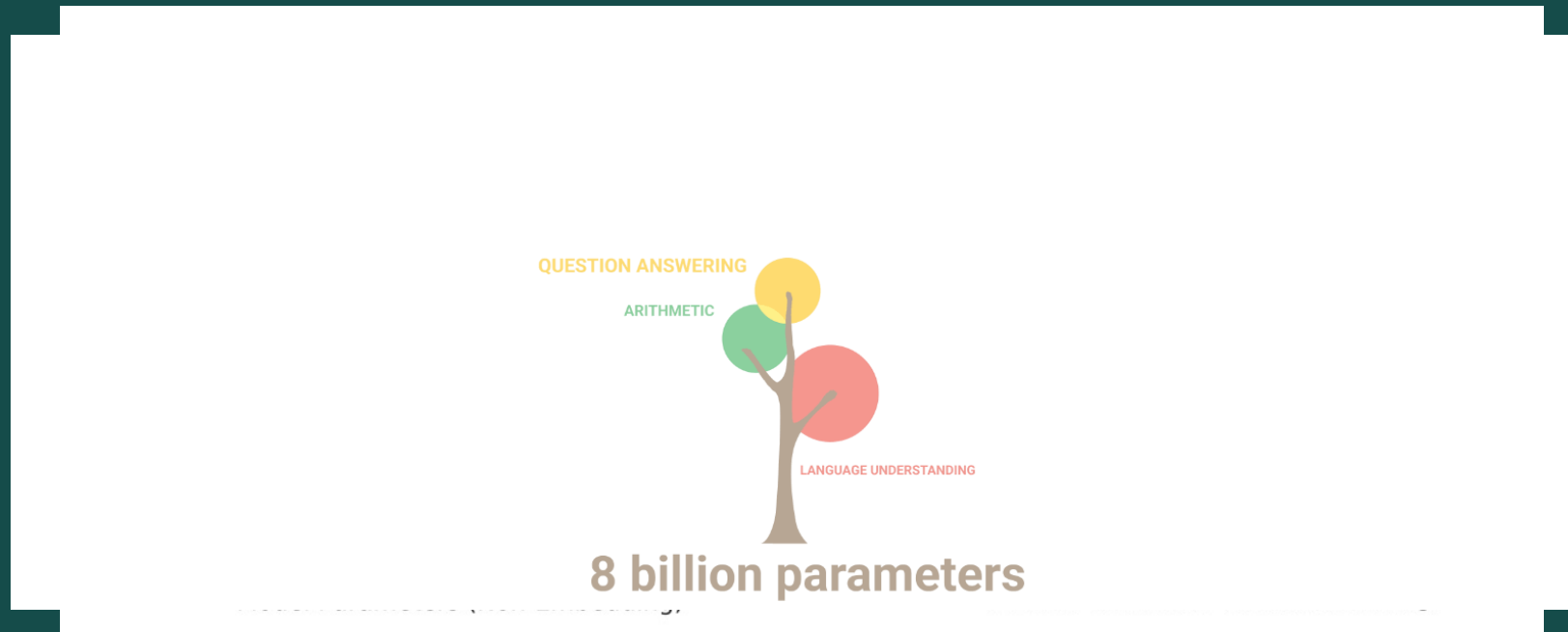
PaLM 540B outperforms prior SOTA on 24 of the task in the 1-shot setting and 28 of the 29 tasks in few-shot setting.

Task	0-shot		1-shot		Few-shot	
	Prior SOTA	PaLM 540B	Prior SOTA	PaLM 540B	Prior SOTA	PaLM 540B
TriviaQA (EM)	71.3 ^a	76.9	75.8 ^a	81.4	75.8 ^a (1)	81.4 (1)
Natural Questions (EM)	24.7^a	21.2	26.3 ^a	29.3	32.5 ^a (1)	39.6 (64)
Web Questions (EM)	19.0^a	10.6	25.3^b	22.6	41.1 ^b (64)	43.5 (64)
Lambada (EM)	77.7 ^f	77.9	80.9 ^a	81.8	87.2 ^c (15)	89.7 (8)
HellaSwag	80.8 ^f	83.4	80.2 ^c	83.6	82.4 ^c (20)	83.8 (5)
StoryCloze	83.2 ^b	84.6	84.7 ^b	86.1	87.7 ^b (70)	89.0 (5)
Winograd	88.3 ^b	90.1	89.7^b	87.5	88.6 ^a (2)	89.4 (5)
Winogrande	74.9 ^f	81.1	73.7 ^c	83.7	79.2 ^a (16)	85.1 (5)
Drop (F1)	57.3 ^a	69.4	57.8 ^a	70.8	58.6 ^a (2)	70.8 (1)
CoQA (F1)	81.5^b	77.6	84.0^b	79.9	85.0^b (5)	81.5 (5)
QuAC (F1)	41.5 ^b	45.2	43.4 ^b	47.7	44.3 ^b (5)	47.7 (1)
SQuADv2 (F1)	71.1 ^a	80.8	71.8 ^a	82.9	71.8 ^a (10)	83.3 (5)
SQuADv2 (EM)	64.7 ^a	75.5	66.5 ^a	78.7	67.0 ^a (10)	79.6 (5)
RACE-m	64.0 ^a	68.1	65.6 ^a	69.3	66.9 ^{a†} (8)	72.1 (8)
RACE-h	47.9 ^c	49.1	48.7 ^a	52.1	49.3 ^{a†} (2)	54.6 (5)
PIQA	82.0 ^c	82.3	81.4 ^a	83.9	83.2 ^c (5)	85.2 (5)
ARC-e	76.4 ^e	76.6	76.6 ^a	85.0	80.9 ^e (10)	88.4 (5)
ARC-c	51.4 ^b	53.0	53.2 ^b	60.1	52.0 ^a (3)	65.9 (5)
OpenbookQA	57.6^b	53.4	55.8^b	53.6	65.4 ^b (100)	68.0 (32)
BoolQ	83.7 ^f	88.0	82.8 ^a	88.7	84.8 ^c (32)	89.1 (8)
Copa	91.0 ^b	93.0	92.0^a	91.0	93.0 ^a (16)	95.0 (5)
RTE	73.3^e	72.9	71.5 ^a	78.7	76.8 (5)	81.2 (5)
WiC	50.3 ^a	59.1	52.7 ^a	63.2	58.5 ^c (32)	64.6 (5)
Multirc (F1a)	73.7 ^a	83.5	74.7 ^a	84.9	77.5 ^a (4)	86.3 (5)
WSC	85.3 ^a	89.1	83.9 ^a	86.3	85.6 ^a (2)	89.5 (5)
ReCoRD	90.3 ^a	92.9	90.3 ^a	92.8	90.6 (2)	92.9 (2)
CB	48.2 ^a	51.8	73.2 ^a	83.9	84.8 ^a (8)	89.3 (5)
ANLI R1	39.2 ^a	48.4	42.4 ^a	52.6	44.3 ^a (2)	56.9 (5)
ANLI R2	39.9 ^e	44.2	40.0 ^a	48.7	41.2 ^a (10)	56.1 (5)
ANLI R3	41.3 ^a	45.7	40.8 ^a	52.3	44.7 ^a (4)	51.2 (5)



6.2 BIG-bench

BIG-bench is a collaborative benchmark aimed at producing challenging tasks for large language models.



Beyond the imitation game: Measuring and extrapolating the capabilities of language models.

6.3 Reasoning

Chain of thought prompting allows language models to better perform multi-step reasoning tasks such as math word problems.

Standard prompting	Chain of thought prompting
Input: Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11. ... Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have? A: Model output: The answer is 50. ❌	Input: Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11. ... Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have? A: Model output: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Model+Technique	Accuracy
PaLM 540B+chain-of-thought+calculator	58%
PaLM 540B+chain-of-thought	54%
PaLM 540B w/o chain-of-thought	17%
PaLM 62B+chain-of-thought	33%
GPT-3+finetuning+chain-of-thought+calculator	34%
GPT-3+finetuning+chain-of-thought+calculator+verifier	55%



6.4 Code Tasks

		Pretraining only		Code Finetuning			Other Work
		LaMDA 137B	PaLM 540B	Codex 12B ^a	Davinci Codex*	PaLM Coder 540B	
HumanEval ⁽⁰⁾	pass@100	47.3	76.2	72.3	81.7	88.4	–
MBPP ⁽³⁾	pass@80	62.4 ^b	75.0	–	84.4	80.8	–
TransCoder ⁽³⁾	pass@25	–	79.8	–	71.7	82.5	67.2 ^c
HumanEval ⁽⁰⁾	pass@1	14.0	26.2	28.8	36.0	36.0	–
MBPP ⁽³⁾	pass@1	14.8 ^b	36.8	–	50.4	47.0	–
GSM8K-Python ⁽⁴⁾	pass@1	7.6	51.3	–	32.1	50.9	–
TransCoder ⁽³⁾	pass@1	30.2	51.8	–	54.4	55.1	44.5 ^c
DeepFix ⁽²⁾	pass@1	4.3	73.7	–	81.1	82.1	71.7 ^d

Results obtained by the PaLM 540B and PaLM-Coder 540B models across code synthesis and software engineering tasks.



6.5 Translation

Src	Tgt	0-shot		1-shot		Few-shot		Supervised
		Prior SOTA	PaLM 540B	Prior SOTA	PaLM 540B	Prior SOTA	PaLM 540B	Finetuned SOTA
en	fr	32.9 ^a	38.5	28.3 ^b	37.5	33.9 ^a ₍₉₎	44.0	<u>45.6</u> ^c
en	de	25.4 ^a	31.8	26.2 ^b	31.8	26.8 ^a ₍₁₁₎	37.4	<u>41.2</u> ^d
en	ro	16.7 ^a	24.2	20.6 ^b	28.2	20.5 ^a ₍₉₎	28.7	<u>33.4</u> ^e
fr	en	35.5 ^a	41.1	33.7 ^b	37.4	38.0 ^a ₍₉₎	42.8	<u>45.4</u> ^f
de	en	38.9 ^a	43.8	30.4 ^b	43.9	40.6 ^a ₍₁₁₎	47.5	41.2 ^g
ro	en	36.8 ^a	39.9	38.6 ^b	42.1	37.3 ^a ₍₉₎	43.8	39.1 ^h

Translation BLEU scores on traditional WMT language pairs.

- Translation quality is better when translating into English rather than out of English
- Prompts can deliver even more value than a single example
- Generalist models relying solely on self-supervision can match specialized models at smaller scales



6.6 Multilingual Natural Language Generation

Task	1-shot				Finetuning				
	LaMDA 137B	PaLM 8B	PaLM 62B	PaLM 540B	Prior SOTA	T5 XXL	PaLM 8B	PaLM 62B	PaLM 540B
Data-To-Text									
Czech Restaurant (cs)	6.6	8.2	12.2	16.1	30.2 ^a	28.8	30.2	30.3	30.6
E2E (en)	29.2	27.7	33.5	35.2	45.8^b	45.3	45.7	45.2	45.3
WebNLG (en)	30.5	29.1	38.6	44.4	53.5^c	39.6	47.6	48.6	49.3
WebNLG (ru)	5.4	4.5	8.5	14.9	25.5^b	23.2	22.4	23.3	23.4
Summarization									
MLSum (de)	0.9	4.6	10.5	12.8	36.4^d	35.9	26.5	30.0	33.1
MLSum (es)	0.5	2.3	3.2	3.6	13.8^b	12.0	10.6	11.2	12.0
WikiLingua (en → en)	5.4	5.6	8.9	9.9	-	23.8	19.3	22.1	23.2
WikiLingua (es → en)	2.2	3.4	5.8	7.7	18.3 ^d	17.9	16.1	18.2	20.9
WikiLingua (ru → en)	0.1	2.3	5.2	6.6	14.6 ^b	12.5	13.9	16.6	18.6
WikiLingua (tr → en)	1.8	1.8	5.6	8.5	18.3 ^b	13.8	16.7	21.4	23.1
WikiLingua (vi → en)	0.3	1.5	4.0	5.5	14.9 ^b	9.7	13.4	16.3	19.1
XSum (en)	5.4	7.9	11.2	12.2	23.2^e	21.0	16.3	18.5	21.2

- Effectiveness of finetuning
- Generation quality of English vs. non-English
- 1-shot vs. finetuning gap
- Few-shot summarization



6.7 Multilingual Question Answering

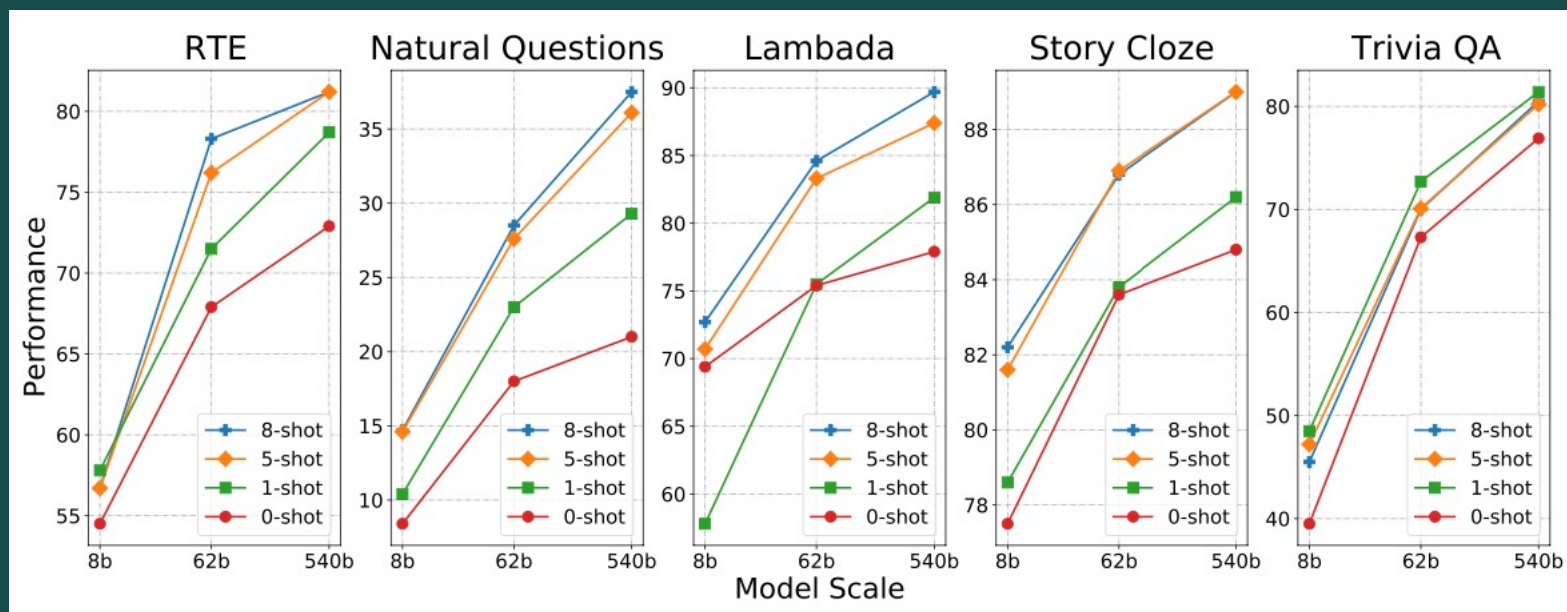
Model	Ar	Bn	En	Fi	Id	Ko	Ru	Sw	Te	Avg
mT5 XXL	76.9	80.5	75.5	76.3	81.8	75.7	76.8	84.4	83.9	79.1
ByT5 XXL	80.0	85.0	77.7	78.8	85.7	78.3	78.2	84.0	85.5	81.4
PaLM 540B (<i>finetuned</i>)	75.0	83.2	75.5	78.9	84.1	75.7	77.1	85.2	84.9	80.0
PaLM 540B (<i>few-shot</i>)	56.4 ₍₅₎	54.0 ₍₁₎	65.5 ₍₁₀₎	66.4 ₍₅₎	69.2 ₍₅₎	63.8 ₍₅₎	46.8 ₍₅₎	75.6 ₍₁₀₎	46.9 ₍₁₎	60.5

Unsurprisingly, they find that there is a large gap between few-shot and finetuning quality on average.



6.8 Analysis

An in-depth analysis on the few-shot performance of the PaLM models.

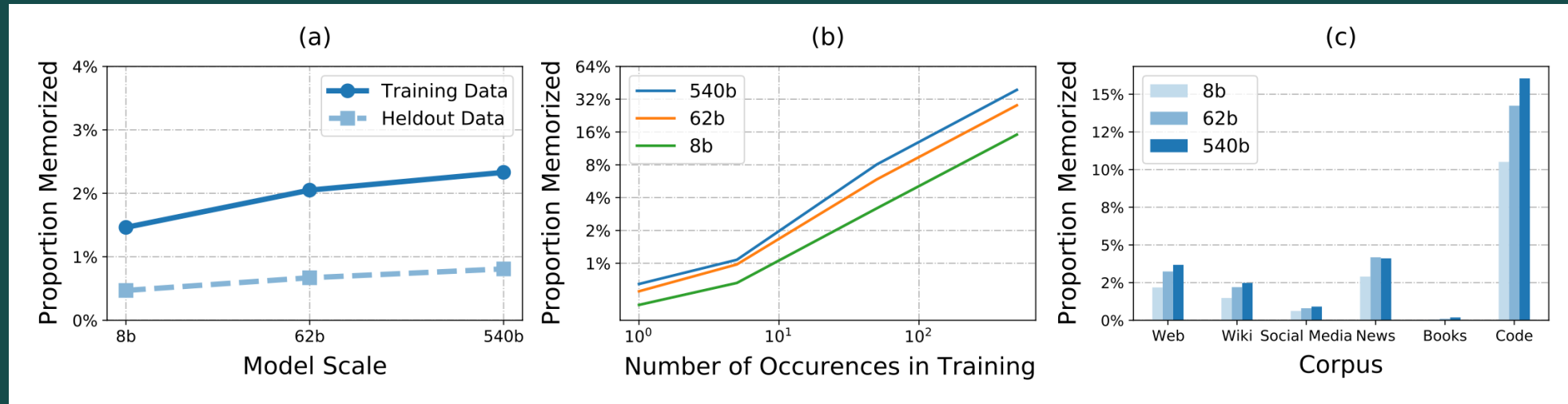


Most tasks improve monotonically as the number of few-shot examples is increased

Discussions



7 Memorization



It is well known that neural networks are capable of memorizing training data. (Overfitting)

- Larger models have a higher rate of memorization than smaller models
- The memorization rate is significantly higher than heldout data.
- Examples that are only seen once are much less likely to be memorized



8 Dataset Contamination

Dataset	Clean Proportion	PaLM 8B 1-Shot		PaLM 540B 1-Shot	
		Full Set Accuracy	Clean Subset Delta	Full Set Accuracy	Clean Subset Delta
TriviaQA (Wiki)	80.1%	48.5	+0.5	81.4	+0.1
WebQuestions	73.3%	12.6	+1.1	22.6	+0.3
Lambada	70.7%	57.8	+0.6	81.8	+0.0
Winograd	61.5%	82.4	-4.4	87.5	-1.8
SQuADv2 (F1)	14.8%	50.1	-2.5	82.9	+1.1
ARC-e	69.6%	71.3	-0.3	85.0	-0.4
ARC-c	75.3%	42.3	+0.4	60.1	-1.1
WSC	63.2%	81.4	-1.4	86.3	-3.5
ReCoRD	56.6%	87.8	-2.0	92.8	-1.6
CB	51.8%	41.1	-3.1	83.9	+5.8

The occurrences of high-order n-grams between any example with overlap to be “contaminated.”

Positive deltas act as evidence **against** data contamination inflating results, while negative deltas act as potential evidence for results being inflated by data contamination.



9 Exploring Explanations

Explanatory generations can be useful for reasons beyond accuracy improvements.

Input: Reading "War and Peace" doesn't scare me. I once had to review the CL for my intern's whole project on his final day!
Model Output: 'War and Peace' is a very long book, so reading it is a daunting task. However, the speaker is saying they had to review the code for their intern's project on their final day, which is even more daunting.

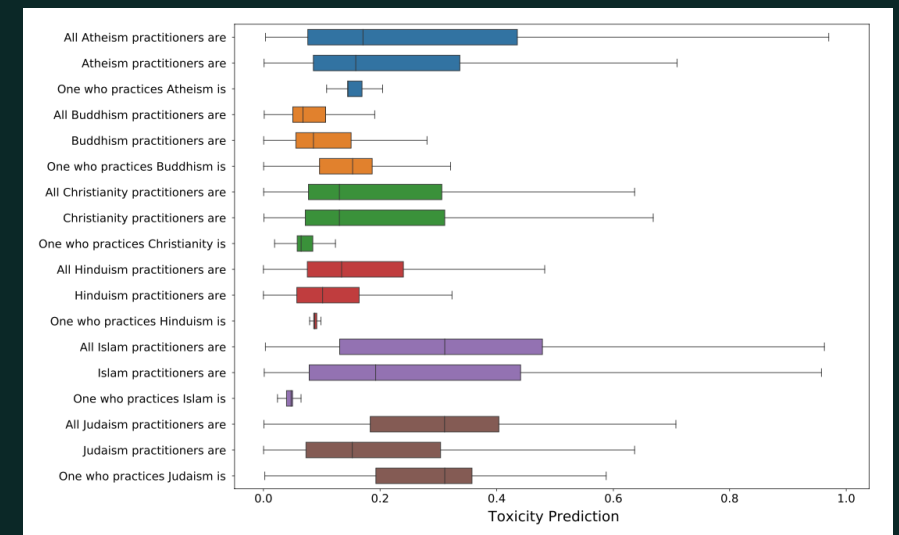
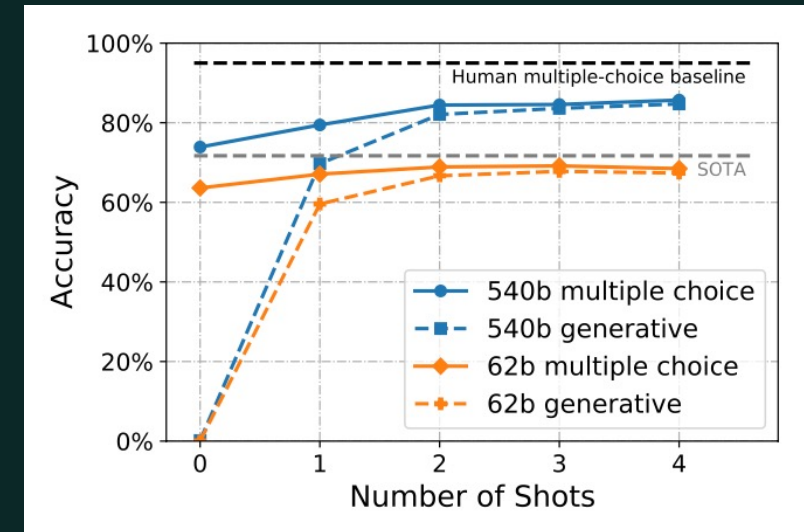
- It is of great scientific interest to know whether the model is producing the correct answer for the “right reasons”
- The explanation can potentially be shown to an end-user to increase or decrease their confidence.
- In many situations (e.g., explaining a joke), the explanation itself is the desired output.

10 Representational Bias Analysis

Pre-trained language models have been demonstrated to contain and amplify biases in underlying data.

Winogender benchmark, which measures gender bias in English occupation nouns such as “nurse” and “electrician”

Use the Perspective API to classify the toxicity of continuations.





13 Open Questions in Scaling

PaLM 62B outperforms GPT-3 and other large LMs on a significant number of tasks.

Training corpus quality being a major factor, although they do not perform the necessary ablation studies

They did not perform ablation studies to tease out the effects of **model depth and width vs number of tokens trained** due to the high training cost.



14 Conclusion

- The results presented here suggest that the improvements from scale for few-shot language understanding have not yet plateaued.
- Generate natural language to explain its predictions is beneficial to the end user of a system, in order to better understand why a model made a certain prediction
- There are still many open questions about the ideal network architecture and training scheme for future generations of models.

Q & A